Aufgabe am Nachmittag: Terraform-Module, Import und Ansible

Tag 9 – Terraform-Module und Ansible

- **Dauer:** ca. 3,5 Stunden (zwei Themen, inkl. Challenge und Aufräumen)
- **Arbeitsform:** Einzel- oder Partnerarbeit
- **Voraussetzungen:** Repository mit `infra/` aus Tag 7 (Terraform mit S3-Backend), Zugang zum **AWS-Sandbox-Konto** und die OIDC-Rolle mit Terraform-Rechten aus Tag 7, Python 3.10 oder neuer, Git installiert **und für Push eingerichtet**
- **Abgabe:** Repository-Link, Modul (Dateien) und Modul-Test, Nachweis des Imports (`terraform state list`), Playbook und Rolle, **zwei Logs** des Playbook-Laufs (Idempotenz), Workflow-Dateien, ausgefüllte Tabelle der Challenge, Nachweis über das Aufräumen

Das baust du heute

Zwei Lücken werden geschlossen: Deine Terraform-Konfiguration wird **wiederverwendbar** (Modul) und **übernimmt vorhandene AWS-Ressourcen** in den State (Import). Danach lernst du das Werkzeug, das Terraform bewusst nicht ersetzt: **Ansible konfiguriert Systeme** – idempotent, agentenlos und aus derselben Pipeline.

Mindestanforderungen (Abgabe-Checkliste):

- ☐ Modul `modules/bucket/` mit `main.tf`, `variables.tf`, `outputs.tf`
- ☐ Kein Provider-Block im Modul, `required_providers` nur im Root
- ☐ Modul im Root über `module` eingebunden, Ein- und Ausgaben genutzt
- ☐ Modul-Test (`terraform test`) läuft grün – ohne Cloud-Zugang
- ☐ Vorhandene Ressource per `import`-Block übernommen (plan zeigt „will be imported“)
- ☐ Backend und Sperre waren vor dem Import aktiv
- ☐ `terraform state list` zeigt die importierte Ressource
- ☐ Playbook für `localhost` mit mindestens drei Tasks
- ☐ **Idempotenz belegt:** zweiter Lauf zeigt `changed=0`
- ☐ Rolle und `requirements.yml` angelegt
- ☐ Playbook läuft in GitHub Actions (gepinnte Version, Syntax-Check)
- ☐ Kein Secret im Log (`no_log`, Vault-Passwort nur über Datei)
- ☐ Challenge gelöst: alle sechs Fehler dokumentiert und behoben

Deine Werte

Wert	Dein Wert
Repository (`owner/repo`)	
AWS-Region	
Role-ARN (Tag 6/7)	
`APP_BUCKET` (von Terraform verwaltet)	

Wert	Dein Wert
`STATE_BUCKET` (nur Bootstrap)	
Name des „Altbestands“-Buckets (Import)	z. B. `mein-projekt-legacy-<deinname>`
Repository-Variable `LEGACY_BUCKET`	derselbe Name – für den Workflow
Name des Vault-Secrets im Repository	`ANSIBLE_VAULT_PASSWORD`

Zeitplan (Vorschlag)

Teil	Schritt	Zeit
A	1 Modul anlegen	25 min
A	2 Modul im Root verwenden	20 min
A	3 Modul testen	20 min
A	4 Vorhandene Ressource importieren	30 min
A	5 Module und Tests in die Pipeline	20 min
B	6 Ansible vorbereiten	15 min
B	7 Erstes Playbook und Idempotenz	30 min
B	8 Rolle und Collections	20 min
B	9 Playbook in GitHub Actions	25 min
B	10 Secrets mit Vault	15 min
B	11 Terraform und Ansible verbinden	15 min
–	Selbstlernzeit + Challenge	45 min

Teil A – Terraform: Module und Import

Schritt 1 – Modul anlegen

Lege unter `infra/modules/bucket/` drei Dateien an.

`infra/modules/bucket/main.tf`

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
    }
  }
}
```

```
resource "aws_s3_bucket" "this" {
  bucket = var.bucket_name
  tags   = var.tags
}
```

infra/modules/bucket/variables.tf

```
variable "bucket_name" {
  type        = string
  description = "Weltweit eindeutiger Name des Buckets"
}

variable "tags" {
  type        = map(string)
  description = "Tags für alle Ressourcen des Moduls"
  default     = {}
}
```

infra/modules/bucket/outputs.tf

```
output "bucket_arn" {
  value       = aws_s3_bucket.this.arn
  description = "ARN des Buckets"
}

output "bucket_name" {
  value       = aws_s3_bucket.this.bucket
  description = "Name des Buckets"
}
```

✅ **Checkpoint:** Die drei Dateien liegen im Modulordner, das Modul **deklariert** seinen Provider (`required_providers`), **konfiguriert** ihn aber nicht.

> **Warum kein Provider im Modul?** Ein Modul mit eigenem Provider-Block ist nicht wiederverwendbar – es bestimmt selbst, wo es läuft. Der Provider gehört ins Root-Modul.

Schritt 2 – Modul im Root verwenden

Ersetze in `infra/main.tf` die direkte Bucket-Ressource durch den Modulaufruf:

```
module "app_bucket" {
  source = "./modules/bucket"

  bucket_name = var.bucket_name

  tags = {
```

```
    Project    = "mein-projekt"
    ManagedBy  = "terraform"
    Repository = var.repository
  }
}
```

Wichtig – sonst wird der Bucket neu gebaut: Für Terraform ändert sich die Adresse der Ressource (von `aws_s3_bucket.app` zu `module.app_bucket.aws_s3_bucket.this`). Ohne Hinweis würde der Plan „destroy and then create“ zeigen. Ein `moved`-Block sagt Terraform, dass es dieselbe Ressource ist:

```
moved {
  from = aws_s3_bucket.app
  to   = module.app_bucket.aws_s3_bucket.this
}
```

Passe die Ausgaben an. Die Ausgaben aus Tag 7 zeigen noch auf die alte Ressource (`aws_s3_bucket.app`) – das ist nach dem Umbau ein Fehler (`Reference to undeclared resource`). `infra/outputs.tf` muss jetzt so aussehen:

```
output "bucket" {
  value = module.app_bucket.bucket_name
}

output "bucket_arn" {
  value = module.app_bucket.bucket_arn
}

output "bucket_name" {
  value = module.app_bucket.bucket_name
}
```

Prüfe lokal (ohne Cloud-Zugang, wie in Tag 7 gelernt):

```
terraform -chdir=infra fmt -check -recursive
terraform -chdir=infra init -backend=false
terraform -chdir=infra validate
```

✅ **Checkpoint:** `fmt -check -recursive` prüft auch das Modul, `init` lädt es, `validate` meldet keine Fehler. Im Plan der Pipeline steht bei der Ressource **„has moved to“** und darunter `No changes` – der Bucket bleibt also bestehen.

> `moved` **statt** `state mv`**:** Der Block steht in der Konfiguration (versioniert und im Plan sichtbar) und wird nach dem Apply zu einer Notiz. Genau dafür gibt es ihn: umbauen ohne zerstören.

> **Wichtig:** Nach dem Hinzufügen eines Moduls muss `terraform init` erneut laufen – sonst: `Error: Module not installed`.

Schritt 3 – Modul testen

Lege `infra/modules/bucket/tests/bucket.tftest.hcl` an:

```
mock_provider "aws" {}

run "bucket_name_wird_gesetzt" {
  command = plan

  variables {
    bucket_name = "test-bucket"
  }

  assert {
    condition     = aws_s3_bucket.this.bucket == "test-bucket"
    error_message = "Der Bucket-Name wird nicht aus der Variable gesetzt"
  }
}
```

Führe den Test aus (der Modulordner wird vorher initialisiert, damit Terraform den Provider kennt):

```
terraform -chdir=infra/modules/bucket init
terraform -chdir=infra/modules/bucket test
```

✅ **Checkpoint:** Der Test läuft grün – **ohne** AWS-Zugangsdaten und ohne Kosten.

> **Warum im Modulordner?** Die Testdatei greift auf die Ressourcen des Moduls zu (`aws_s3_bucket.this`). Im Modulordner ist das Modul selbst das Root-Modul – deshalb funktioniert der Test dort. `mock_provider` ersetzt AWS durch einen Mock.

Schritt 4 – Vorhandene Ressource importieren

1. **Altbestand bereitstellen:** Lege einmalig per Hand einen Bucket an, den niemand in Terraform verwaltet – z. B. `mein-projekt-legacy-<deinname>`. *(Wenn du den Site-Bucket aus Tag 6 noch hast, kannst du auch diesen nehmen.)*
2. **Ressource und Import-Block schreiben.** Ergänze in `infra/main.tf`:

```
resource "aws_s3_bucket" "legacy" {
  bucket = var.legacy_bucket_name
}

import {
```

```
  to = aws_s3_bucket.legacy
  id = var.legacy_bucket_name
}
```

Und in `infra/variables.tf`:

```
variable "legacy_bucket_name" {
  type        = string
  description = "Name des per Hand angelegten Buckets (Altbestand)"
}
```

3. **Lokal prüfen** (ohne AWS-Zugangsdaten, wie in Tag 7 gelernt):

```
terraform -chdir=infra fmt -check -recursive
terraform -chdir=infra init -backend=false
terraform -chdir=infra validate
```

4. **Plan in der Pipeline lesen** – lokal hast du keine AWS-Zugangsdaten, also läuft der Plan dort, wo die OIDC-Rolle ist. Committe die Änderung, öffne den Run und lies den Plan (Schritt 5 zeigt den passenden Workflow). Erwartete Ausgabe:

```
Plan: 1 to import, 0 to add, 0 to change, 0 to destroy
```

5. **Anwenden** (im Apply-Job nach deiner Freigabe) und im Log prüfen:

```
terraform -chdir=infra state list
```

✅ **Checkpoint:** `aws_s3_bucket.legacy` steht im State, und der nächste Plan ist leer (`No changes`).

> **Die ID ist die halbe Miete:** Bei einem S3-Bucket ist die ID der **Bucket-Name** – nicht die ARN. Eine falsche ID führt zu `Cannot import non-existent remote object`.

> **Wenn der Plan ein `update in-place` zeigt:** Deine Konfiguration weicht von der echten Ressource ab (typisch: Tags). Passe die Konfiguration an, bis der Plan leer ist – dann gibt es keine Überraschungen.

Schritt 5 – Module und Tests in die Pipeline

Ergänze im Plan-Job aus Tag 7 (`.github/workflows/infra.yml`) diese Steps **vor** dem Plan:

```
      - name: Format prüfen (inkl. Module)
        run: terraform -chdir=infra fmt -check -recursive

      - name: Konfiguration prüfen
        run: terraform -chdir=infra validate

      - name: Modul-Tests
        run: |
          terraform -chdir=infra/modules/bucket init
          terraform -chdir=infra/modules/bucket test

      - name: Plan erzeugen
        run: |
          terraform -chdir=infra plan \
            -var "bucket_name=${{ vars.APP_BUCKET }}" \
            -var "repository=${{ github.repository }}" \
            -var "legacy_bucket_name=${{ vars.LEGACY_BUCKET }}" \
            -out=tfplan -input=false
```

Lege die Repository-Variable `LEGACY_BUCKET` an (der Name aus Schritt 4) und führe den Workflow aus. Der Apply-Job bleibt unverändert: **genau dieser Plan**, nach Freigabe.

✅ **Checkpoint:** Der Lauf ist grün, der Plan zeigt den Import als Teil des Plans, und im Log steht das Testergebnis.

Teil B – Ansible

Schritt 6 – Ansible vorbereiten

```
python -m venv .venv-ansible

# macOS / Linux
source .venv-ansible/bin/activate

# Windows
.venv-ansible\Scripts\activate

python -m pip install "ansible-core==2.21.2"
ansible --version
```

> **Version prüfen:** `ansible-core` wird schnell weiterentwickelt (aktuell 2.21.x). Nimm die Version, die auf https://pypi.org/project/ansible-core/ als aktuell steht, und pinne sie.

Lege die Struktur an:

```
ansible/
├── inventory
├── playbook.yml
├── requirements.yml
└── roles/
```

ansible/inventory

```
[local]
localhost ansible_connection=local
```

✅ **Checkpoint:** `ansible --version` zeigt die gepinnte Version; `ansible-inventory -i ansible/inventory --list` listet `localhost`.

Schritt 7 – Erstes Playbook und Idempotenz

ansible/playbook.yml

```yaml
- name: Webserver einrichten
  hosts: localhost
  connection: local

  tasks:
    - name: Verzeichnis anlegen
      ansible.builtin.file:
        path: /tmp/web
        state: directory
        mode: "0755"

    - name: Index schreiben
      ansible.builtin.copy:
        content: "Hallo aus Ansible\n"
        dest: /tmp/web/index.html
      notify: Neu laden

    - name: Konfiguration aus Template
      ansible.builtin.template:
        src: templates/site.conf.j2
        dest: /tmp/web/site.conf
      notify: Neu laden

  handlers:
    - name: Neu laden
      ansible.builtin.debug:
        msg: "Konfiguration hat sich geändert"
```

ansible/templates/site.conf.j2

```
# bucket: {{ bucket_name | default('noch nicht gesetzt') }}
server {
  listen {{ port | default(8080) }};
  root {{ web_root | default('/tmp/web') }};
}
```

Führe das Playbook **zweimal** aus:

```
ansible-playbook -i ansible/inventory ansible/playbook.yml
ansible-playbook -i ansible/inventory ansible/playbook.yml
```

✅ **Checkpoint:** Der erste Lauf zeigt `changed=2` oder mehr, der **zweite Lauf** `changed=0`. Beide Logs sind Teil der Abgabe.

Weitere nützliche Aufrufe:

```
ansible-playbook -i ansible/inventory ansible/playbook.yml --check   # Trockenlauf
ansible-playbook -i ansible/inventory ansible/playbook.yml --diff    # Unterschiede zeigen
```

> **Idempotenz ist die zentrale Idee:** Ein Playbook beschreibt einen **Zustand**, keine Befehlsfolge. Aufgaben, die immer etwas tun (z. B. `shell`), melden bei jedem Lauf `changed` – dann brauchst du `changed_when`, `creates` oder ein zustandsbasiertes Modul.

Schritt 8 – Rolle und Collections

Verschiebe die Aufgaben in eine Rolle:

```
ansible/roles/webserver/
├── tasks/main.yml
├── handlers/main.yml
├── templates/site.conf.j2
└── defaults/main.yml
```

ansible/roles/webserver/tasks/main.yml

```
- name: Verzeichnis anlegen
  ansible.builtin.file:
    path: /tmp/web
```

```
    state: directory
    mode: "0755"

- name: Index schreiben
  ansible.builtin.copy:
    content: "Hallo aus Ansible\n"
    dest: /tmp/web/index.html

- name: Konfiguration aus Template
  ansible.builtin.template:
    src: site.conf.j2
    dest: /tmp/web/site.conf
  notify: Neu laden
```

ansible/roles/webserver/handlers/main.yml

```
- name: Neu laden
  ansible.builtin.debug:
    msg: "Konfiguration hat sich geändert"
```

ansible/roles/webserver/defaults/main.yml

```
web_root: /tmp/web
port: 8080
```

Verschiebe die Datei `site.conf.j2` in das Template-Verzeichnis der Rolle (`ansible/roles/webserver/templates/`). Ergänze außerdem eine Aufgabe, die ein Modul aus der Collection nutzt – damit brauchst du `requirements.yml` wirklich:

```
- name: Anwendungs-Konfiguration schreiben
  community.general.ini_file:
    path: "{{ web_root }}/app.ini"
    section: app
    option: port
    value: "{{ port }}"
    mode: "0644"
```

ansible/requirements.yml

```
collections:
  - name: community.general
    version: "9.0.0"
```

Nutze die Rolle im Playbook:

```
- name: Webserver einrichten
  hosts: localhost
  connection: local
  roles:
    - webserver
```

Installiere die Collections und prüfe:

```
ansible-galaxy collection install -r ansible/requirements.yml
ansible-galaxy collection list | head
```

> **Version prüfen:** Die Versionen von Collections ändern sich regelmäßig – schau im Zweifel auf https://galaxy.ansible.com/ui/ nach und passe `requirements.yml` an.

✅ **Checkpoint:** Das Playbook läuft weiterhin und bleibt idempotent (`changed=0` im zweiten Lauf).

> **Collections liefern Module nach**, die nicht im Ansible-Kern liegen (erkennbar am Namen `community.general.…`). Fehlt die Collection, lautet die Meldung `couldn't resolve module/action`.

Schritt 9 – Playbook in GitHub Actions

Lege `.github/workflows/configure.yml` an:

```
name: Konfiguration

on:
  workflow_dispatch:

jobs:
  configure:
    runs-on: ubuntu-latest
    env:
      ANSIBLE_STDOUT_CALLBACK: yaml
      ANSIBLE_CORE_VERSION: "2.21.2"
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Cache
        uses: actions/cache@v4
        with:
          path: |
            ~/.cache/pip
```

```
            ~/.ansible/collections
          key: ${{ runner.os }}-ansible-${{
hashFiles('ansible/requirements.yml') }}

      - name: Ansible installieren
        run: |
          python -m pip install --upgrade pip
          python -m pip install "ansible-core==${{ env.ANSIBLE_CORE_VERSION
}}"
          ansible-galaxy collection install -r ansible/requirements.yml

      - name: Syntax prüfen
        run: ansible-playbook --syntax-check -i ansible/inventory
ansible/playbook.yml

      - name: Playbook ausführen
        run: ansible-playbook -i ansible/inventory ansible/playbook.yml
```

Ergänze zum Nachweis der Idempotenz einen **zweiten Playbook-Lauf im selben Job**:

```
      - name: Idempotenz prüfen (zweiter Lauf)
        run: ansible-playbook -i ansible/inventory ansible/playbook.yml
```

Starte den Workflow.

✅ **Checkpoint:** Der Workflow ist grün, und der **zweite Lauf im selben Job** zeigt `changed=0`.

> **Wichtig:** In der Pipeline startet jeder Workflow-Lauf auf einem **frischen Runner** – zwei separate Workflow-Läufe zeigen deshalb beide `changed>0`. Der Idempotenz-Nachweis funktioniert nur mit zwei Läufen **innerhalb desselben Jobs** (oder auf einem echten Server, der bestehen bleibt).

> **Optionale Qualitätsstufe:** `ansible-lint` als eigener Step (`python -m pip install ansible-lint`, dann `ansible-lint ansible/playbook.yml`). Es bringt eine passende `ansible-core`-Version mit – installiere es deshalb **nach** `ansible-core` und prüfe die Version.

Schritt 10 – Secrets mit Vault

1. Lege im Repository das Secret `ANSIBLE_VAULT_PASSWORD` an (Fantasiewert).
2. Verschlüssle einen Wert lokal:

```
printf '%s' "mein-geheimnis" > vault-pass.txt
ansible-vault encrypt_string \
  --vault-password-file vault-pass.txt \
  "geheimer-wert" --name api_token
```

3. Nutze das Ergebnis in einer Variablendatei (`ansible/vars/secrets.yml`) und im Playbook:

```
- name: API aufrufen
  ansible.builtin.debug:
    msg: "Token ist gesetzt"
  no_log: true
  when: api_token is defined
```

4. Ergänze im Workflow die Vault-Übergabe **und** das Aufräumen:

```
      - name: Vault-Passwort bereitstellen
        run: printf '%s' "${{ secrets.ANSIBLE_VAULT_PASSWORD }}" > vault-
pass.txt

      - name: Playbook ausführen
        run: |
          ansible-playbook -i ansible/inventory ansible/playbook.yml \
            --vault-password-file vault-pass.txt \
            -e @ansible/vars/secrets.yml

      - name: Aufräumen
        if: always()
        run: rm -f vault-pass.txt
```

✅ **Checkpoint:** Der Lauf ist grün, im Log steht **kein** Klartext-Wert, und `vault-pass.txt` existiert nach dem Lauf nicht mehr.

> **Zwei Fallen:** Schreibe das Passwort mit `printf '%s'` (ohne Zeilenumbruch) – sonst schlägt die Entschlüsselung fehl. Und räume mit `if: always()` auf, damit die Datei auch bei Fehlern verschwindet.

Schritt 11 – Terraform und Ansible verbinden

Die Übergabe läuft über die Terraform-Outputs. Ergänze im Terraform-Job (nach dem Apply):

```
      - name: Outputs bereitstellen
        run: |
          terraform -chdir=infra output -raw bucket_name > tf-bucket.txt

      - name: Outputs als Artifact
        uses: actions/upload-artifact@v4
        with:
          name: tf-outputs
          path: tf-bucket.txt
```

Und im Ansible-Job:

```
    needs: apply
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: tf-outputs

      - name: Playbook mit Terraform-Wert
        run: |
          ansible-playbook -i ansible/inventory ansible/playbook.yml \
            --extra-vars "bucket_name=$(cat tf-bucket.txt)"
```

✅ **Checkpoint:** In `/tmp/web/site.conf` steht in der ersten Zeile der Bucket-Name aus Terraform – die beiden Werkzeuge arbeiten in einer Kette, nicht nebeneinander.

> **Wer darf was?** Der Terraform-Job braucht AWS-Rechte (OIDC). Der Ansible-Job braucht nur die Zugangsdaten, die er für die Zielsysteme hat – hier keine.

Selbstlernzeit

- Sieh dir das Terraform-Test-Framework genauer an (`.tftest.hcl`, `command = apply`, `expect_failures`) und notiere, welchen Test du für dein Modul noch ergänzen würdest.
- Prüfe die Struktur-Empfehlungen für Module (https://developer.hashicorp.com/terraform/language/modules/develop/structure) und vergleiche sie mit deinem Modul.
- Lies die Idempotenz-Empfehlungen für Ansible und suche in deinem Playbook nach Stellen, die bei jedem Lauf `changed` melden.
- Schau dir `ansible-lint` an: Welche Regeln würden dein Playbook heute verletzen?

Challenge: sechs Fehler debuggen

Drei Fehler stecken im Terraform-Teil, drei im Ansible-Teil.

`infra/main.tf`

```
module "app_bucket" {
  source     = "./modules/bucket"
  bucketname = var.bucket_name
}

resource "aws_s3_bucket" "legacy" {
  bucket = var.legacy_bucket_name
}

import {
  to = aws_s3_bucket.legacy
  id = "arn:aws:s3:::mein-projekt-legacy"
}
```

```
output "bucket_arn" {
  value = module.app_bucket.bucket_id
}
```

`.github/workflows/configure.yml` **(Auszug)**

```
      - name: Collections
        run: ansible-galaxy collection install -r ansible/requirements.yml

      - name: Playbook
        run: ansible-playbook -i ansible/inventory ansible/playbook.yml
```

`ansible/requirements.yml`

```
collections:
  - name: ansible.posix
    version: "2.0.0"
```

`ansible/roles/webserver/tasks/main.yml` **(Auszug)**

```
- name: Anwendungs-Konfiguration schreiben
  community.general.ini_file:
    path: /tmp/web/app.ini
    section: app
    option: port
    value: "8080"
    mode: "0644"

- name: Statuszeile anfügen
  ansible.builtin.shell: echo "$(date)" >> /tmp/web/status.log
```

Deine Aufgabe

1. Finde alle **sechs** Fehler.
2. Notiere zu jedem: **Meldung oder Risiko**, **Ursache**, **Fix**.
3. Repariere alles, bis Terraform grün und der zweite Ansible-Lauf idempotent ist.

Meldung / Risiko	Ursache	Fix

✅ **Checkpoint:** Sechs Fehler dokumentiert, Terraform-Plan leer, Playbook zweimal ohne Änderung – und kein Secret im Log.

Häufige Fehler und ihre Ursachen

Symptom	Ursache	Lösung
`Error: Module not installed`	Modul neu hinzugefügt, `init` fehlt	`terraform init` (in CI: der init-Step)
`Error: Unsupported argument`	Argument im Modulaufruf falsch geschrieben oder nicht deklariert	Namen in `variables.tf` und im Aufruf abgleichen
`Error: Unsupported attribute`	Ausgabe gibt es nicht (oder falscher Name)	Namen in `outputs.tf` prüfen: `module.x.<output>`
`Cannot import non-existent remote object`	Falsche Import-ID (z. B. ARN statt Name)	ID-Format der Ressource prüfen
Nach dem Import: `update in-place` oder `replace`	Konfiguration weicht von der echten Ressource ab	Attribute angleichen (Tags!), dann erneut planen
`Attempting to decrypt but no vault secrets found`	Vault-Passwort nicht übergeben	`--vault-password-file` ergänzen, Datei mit `printf` schreiben
`couldn't resolve module/action`	Collection fehlt	Modul in `requirements.yml` ergänzen und installieren
`changed` bei jedem Lauf	Shell-Task ohne Idempotenz	`changed_when`/`creates` oder zustandsbasiertes Modul
`Missing sudo password`	`become` ohne Rechte	Im Runner ohne `become` arbeiten oder Rechte klären
`Syntax Error while loading YAML`	Einrückung oder fehlendes `-`	`ansible-playbook --syntax-check` nutzen
Secrets im Log	Aufgabe ohne `no_log`	`no_log: true` setzen, Werte nie ausgeben
Playbook läuft lokal, nicht in CI	Collection oder Inventory fehlt	Cache/Install-Steps prüfen, `ansible-inventory --list`

Abgabe

- [ ] Repository-Link mit `infra/modules/bucket/` und `ansible/`
- [ ] Modul testet sich selbst (`terraform test` grün, Log oder Screenshot)
- [ ] Nachweis des Imports (`terraform state list` zeigt die Ressource)
- [ ] Plan-Job mit `fmt -check -recursive`, `validate`, Modul-Test und Artifact
- [ ] Playbook mit Rolle und `requirements.yml`
- [ ] **Zwei Log-Ausschnitte** des Playbook-Laufs (`changed>0`, dann `changed=0`)
- [ ] Workflow `configure.yml` läuft grün, mit gepinnter Ansible-Version

- [ ] Vault-Passwort über Secret, `no_log`, Aufräumen mit `if: always()`
- [ ] Verbindung Terraform → Ansible über Outputs belegt
- [ ] Ausgefüllte Tabelle der Challenge (sechs Fehler)
- [ ] Keine Secret-Werte in Logs, README oder Commits

Wenn du früher fertig bist:

- Ergänze einen zweiten Test mit `command = apply` und `mock_provider`.
- Baue ein zweites Modul (z. B. `modules/bucket_policy`) und nutze den Output des ersten Moduls.
- Konfiguriere eine echte EC2-Instanz: Terraform legt sie an, Ansible konfiguriert sie per SSH.
- Führe `ansible-lint` ein und behebe die wichtigsten Hinweise.

Anhang A – Lösungen

> ⚠️ **Erst nach der Challenge lesen.**

A1 Auflösung der Challenge

Meldung / Risiko	Ursache	Fix
`Error: Unsupported argument` – `bucketname`	Argumentname falsch geschrieben; das Modul deklariert `bucket_name`	Aufruf korrigieren: `bucket_name = var.bucket_name`
`Cannot import non-existent remote object`	Import-ID ist die ARN, für einen Bucket erwartet Terraform den **Namen**	`id = "mein-projekt-legacy"` (Bucket-Name)
`Error: Unsupported attribute` – `bucket_id`	Das Modul gibt `bucket_arn` und `bucket_name` aus, nicht `bucket_id`	`module.app_bucket.bucket_arn` verwenden oder die Ausgabe im Modul ergänzen
`Attempting to decrypt but no vault secrets found`	Das Playbook nutzt verschlüsselte Variablen, aber das Vault-Passwort wird nicht übergeben	`--vault-password-file vault-pass.txt` ergänzen; Passwort aus GitHub Secret mit `printf` schreiben, Aufräumen mit `if: always()`
`couldn't resolve module/action 'community.general.ini_file'`	Die Collection fehlt in `requirements.yml` (dort steht nur `ansible.posix`)	`community.general` mit Version ergänzen und installieren

Meldung / Risiko	Ursache	Fix
Datei wächst bei jedem Lauf, immer `changed`	`shell` mit Umleitung ist nicht idempotent	`ansible.builtin.lineinfile` mit `regexp` nutzen – oder `shell` mit `changed_when: false`/`creates`

A2 Modul (Lösung)

Siehe Schritt 1: `main.tf`, `variables.tf`, `outputs.tf` – ohne Provider-Block, mit beschreibenden Variablen und Ausgaben.

Import zusätzlich absichern (Bootstrap-Ressourcen wie Rolle und OIDC-Provider):

```
resource "aws_iam_role" "github" {
  name               = "github-actions-terraform"
  assume_role_policy = jsonencode({ /* ... */ })

  lifecycle {
    prevent_destroy = true
  }
}
```

A3 Test (Lösung)

```
mock_provider "aws" {}

run "bucket_name_wird_gesetzt" {
  command = plan

  variables {
    bucket_name = "test-bucket"
  }

  assert {
    condition     = aws_s3_bucket.this.bucket == "test-bucket"
    error_message = "Der Bucket-Name wird nicht aus der Variable gesetzt"
  }
}
```

Ausführen: `terraform -chdir=infra/modules/bucket test`.

A4 Playbook und Rolle (Lösung)

Siehe Schritt 7 und 8. Entscheidend für die Bewertung ist der **zweite Lauf**: `changed=0` beweist, dass alle Aufgaben zustandsbasiert sind. Der Handler läuft nur beim ersten Lauf.

A5 Kombination in der Pipeline

```
  apply:
    needs: plan
    if: github.ref == 'refs/heads/main'
    environment: production
    # ... wie in Tag 7
    steps:
      # ...
      - name: Plan anwenden
        run: terraform -chdir=infra apply -input=false tfplan

      - name: Outputs bereitstellen
        run: |
          terraform -chdir=infra output -raw bucket_name > tf-bucket.txt

      - uses: actions/upload-artifact@v4
        with:
          name: tf-outputs
          path: tf-bucket.txt

  configure:
    needs: apply
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with:
          name: tf-outputs
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: |
          python -m pip install "ansible-core==2.21.2"
          ansible-galaxy collection install -r ansible/requirements.yml
      - run: |
          ansible-playbook -i ansible/inventory ansible/playbook.yml \
            --extra-vars "bucket_name=$(cat tf-bucket.txt)"
```

Drei Regeln zum Mitnehmen:

1. **Ein Modul, ein Zweck** – und niemals ein Provider-Block darin.
2. **Import planbar machen:** der `import`-Block steht in der Konfiguration, nicht im Terminal.
3. **Idempotenz beweisen:** zweimal laufen lassen, `changed=0` zeigen.

Anhang B – Nützliche Links

- Terraform – Module entwickeln: https://developer.hashicorp.com/terraform/language/modules/develop

- Terraform – Modul-Struktur: https://developer.hashicorp.com/terraform/language/modules/develop/structure
- Terraform – Import (Blöcke): https://developer.hashicorp.com/terraform/language/import
- Terraform – Tests: https://developer.hashicorp.com/terraform/language/tests
- Ansible – Dokumentation: https://docs.ansible.com/
- Ansible – Playbooks: https://docs.ansible.com/ansible/latest/playbook_guide/index.html
- Ansible – Collections (Galaxy): https://galaxy.ansible.com/ui/
- Ansible – Vault: https://docs.ansible.com/ansible/latest/vault_guide/index.html
- `ansible-lint`: https://ansible.readthedocs.io/projects/lint/